Q1. Define a class called Date which stores a date (day, moth, year). Define appropriate setters and getters and override the __str__ method to print the date in the format dd-mm-yy.

```
class Date():

    def __init__(self,dy=0,m=0,y=0):

        self.day=int(dy)
        self.month=int(m)
        self.year=int(y)

    def setDay(self,dy):
        self.day=int(dy)
    def setMonth(self,m):
        self.month=int(m)
    def setYear(self,y):
        self.year=int(y)
    def getDay(self):
        return self.day
    def getMonth(self):
        return self.month
    def getYear(self):
        return self.year
    def __str__(self):

        if self.day<10:
            day="0"+str(self.day)
        else:
            day= str(self.day)

        if self.month<10:
            month= "0"+str(self.month)
        else:
            month= str(self.month)
        if self.year<10:
            year= "0"+str(self.year)
        else:
            year=str(self.year)[-2:]

        return str(day)+"-"+str(month)+"-"+str(year)


a=Date()
a.setDay(29)
a.setMonth(9)
```

```
a.setYear(2026)
print("\nDate:",end="")
print(a)
```

```
class Date():

    def __init__(self,dy=0,m=0,y=0):

        self.day=int(dy)
        self.month=int(m)
        self.year=int(y)

    def setDay(self,dy):
        self.day=int(dy)
    def setMonth(self,m):
        self.month=int(m)
    def setYear(self,y):
        self.year=int(y)
    def getDay(self):
        return self.day
    def getMonth(self):
        return self.month
    def getYear(self):
        return self.year
    def __str__(self):

        if self.day<10:
            day="0"+str(self.day)
        else:
            day= str(self.day)

        if self.month<10:
            month= "0"+str(self.month)
        else:
            month= str(self.month)
        if self.year<10:
            year= "0"+str(self.year)
        else:
            year=str(self.year)[-2:]

        return str(day)+"-"+str(month)+"-"+str(year)
```

```
a=Date()
a.setDay(29)
a.setMonth(9)
a.setYear(2026)
print("\nDate:",end="")
print(a)
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Shaya\Downloads\Inheritance Assignment\Q1. Define a class called Date which stores a date (day, moth, year). Define appropriate setters and getters and override the __str__ method to print the date in the format dd-mm-yy..py

Date:29-09-26
>>>
```

Q2. Add a member variable format to class Date in problem 1. This variable stores numbers that represents various date formats; for example: if format=1, the date should be printed in dd-mm-yy, if format=2, the date is printed in mm/dd/yy, if format=3 date is printed on mm_name dd, yy. The user of class Date should be able to set a specific format for date printing by the __str__ method.

```
class Date():

    def __init__(self,dy=0,m=0,y=0,f=0):

        self.day=int(dy)
        self.month=int(m)
        self.year=int(y)
        self.format=int(f)

    def setDay(self,dy):
        self.day=int(dy)
    def setMonth(self,m):
        self.month=int(m)
    def setYear(self,y):
        self.year=int(y)
    def getDay(self):
        return self.day
    def getMonth(self):
        return self.month
    def getYear(self):
        return self.year
    def setFormat(self,f):
        self.format=int(f)
    def getFormat(self):
        return self.format
    def __str__(self):
        if self.day<10:
            day="0"+str(self.day)
        else:
            day= str(self.day)

        if self.month<10:
            month= "0"+str(self.month)
        else:
            month= str(self.month)
        if self.year<10:
            year= "0"+str(self.year)
        else:
            year=str(self.year)[-2:]
```

```
        if self.format==1:
            return str(day)+"-"+str(month)+"-"+str(year)


        if self.format==2:
            return str(month)+"/"+str(day)+"/"+str(year)
        if self.format==3:
            if self.month==1:
                month="January"
            elif self.month==2:
                month="February"
            elif self.month==3:
                month="March"
            elif self.month==4:
                month="April"
            elif self.month==5:
                month="May"
            elif self.month==6:
                month="June"
            elif self.month==7:
                month="July"
            elif self.month==8:
                month="August"
            elif self.month==9:
                month="September"
            elif self.month==10:
                month="October"
            elif self.month==11:
                month="November"
            elif self.month==12:
                month="December"

            return str(month)+" "+str(day)+","+" "+str(year)


a=Date()
a.setDay(29)
a.setMonth(9)
a.setYear(2026)
a.setFormat(1)
print("\n\t\t\tFormat 1:")
print("\nDate:",end="")
print(a)
a.setFormat(2)
print("\n\t\t\tFormat 2:")
print("\nDate:",end="")
```

```
        print(a)
        a.setFormat(3)
        print("\n\t\t\tFormat3:")
        print("\nDate:",end="")
        print(a)
```

```
class Date():

    def __init__(self,dy=0,m=0,y=0,f=0):

        self.day=int(dy)
        self.month=int(m)
        self.year=int(y)
        self.format=int(f)

    def setDay(self,dy):
        self.day=int(dy)
    def setMonth(self,m):
        self.month=int(m)
    def setYear(self,y):
        self.year=int(y)
    def getDay(self):
        return self.day
    def getMonth(self):
        return self.month
    def getYear(self):
        return self.year
    def setFormat(self,f):
        self.format=int(f)
    def getFormat(self):
        return self.format
    def __str__(self):
        if self.day<10:
            day="0"+str(self.day)
        else:
            day= str(self.day)

        if self.month<10:
            month= "0"+str(self.month)
        else:
            month= str(self.month)
        if self.year<10:
            year= "0"+str(self.year)
```

```
        else:
            year=str(self.year)[-2:]

        if self.format==1:
            return str(day)+"-"+str(month)+"-"+str(year)


        if self.format==2:
            return str(month)+"/"+str(day)+"/"+str(year)
        if self.format==3:
            if self.month==1:
                month="January"
            elif self.month==2:
                month="February"
            elif self.month==3:
                month="March"
            elif self.month==4:
                month="April"
            elif self.month==5:
                month="May"
            elif self.month==6:
                month="June"
            elif self.month==7:
                month="July"
            elif self.month==8:
                month="August"
            elif self.month==9:
                month="September"
            elif self.month==10:
                month="October"
            elif self.month==11:
                month="November"
            elif self.month==12:
                month="December"

            return str(month)+" "+str(day)+","+" "+str(year)
```

```
a=Date()
a.setDay(29)
a.setMonth(9)
a.setYear(2026)
a.setFormat(1)
print("\n\t\t\tFormat 1:")
print("\nDate:",end="")
print(a)
a.setFormat(2)
print("\n\t\t\tFormat 2:")
print("\nDate:",end="")
print(a)
a.setFormat(3)
print("\n\t\t\tFormat3:")
print("\nDate:",end="")
print(a)
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:\Users\Shaya\Downloads\Inheritance Assignment\Q2.py ========

                        Format 1:

Date:29-09-26

                        Format 2:

Date:09/29/26

                        Format3:

Date:September 29, 26
>>>
```

Q3. Write code to develop a class named Vehicles having three private attributes noOfWheels, color and modelNo; define related constructor and public getter/setter methods. Use appropriate types for all attributes, method parameters and return values wherever needed. Instantiate necessary objects to test the functionality of this class.

```
class Vehicles():
    count=0

    def __init__(self,n=4,c="Red",mN=1987):
        self.__noOfWheels=n
        self.__color=c
        self.__modelNo=mN
    def setNoOfWheels(self,n):
        self.__noOfWheels=n
    def getNoOfWheels(self):
        return self.__noOfWheels
    def setColor(self,c):
        self.__color=c
    def getColor(self):
        return self.__color
    def setModelNo(self,mN):
        self.__modelNo=mN
    def getModelNo(self):
        return self.__modelNo
    def countVehicles(self):
        Vehicles.count+=1
        self.assignId(Vehicles.count)
        return Vehicles.count

    def assignId(self,id):
        self.Id=id
    def getId(self):
        return self.Id

v=Vehicles()
print("\nDefault No. of Wheels",v.getNoOfWheels())
print("Default Color:",v.getColor())
print("Default Model No:",v.getModelNo())
print("\nNumber of Objects in v:",v.countVehicles())
print("ID of v:",v.getId())

v1=Vehicles()
v1.setNoOfWheels(5)
v1.setColor("Black")
v1.setModelNo(2015)
```

```
print("\nSet No.of Wheels:",v1.getNoOfWheels())
print("Set Color:",v1.getColor())
print("Set Model No:",v1.getModelNo())
print("\nNumber of Objects in v1:",v1.countVehicles())
print("ID of v1:",v1.getId())
```

```
class Vehicles():
    count=0

    def __init__(self,n=4,c="Red",mN=1987):
        self.__noOfWheels=n
        self.__color=c
        self.__modelNo=mN
    def setNoOfWheels(self,n):
        self.__noOfWheels=n
    def getNoOfWheels(self):
        return self.__noOfWheels
    def setColor(self,c):
        self.__color=c
    def getColor(self):
        return self.__color
    def setModelNo(self,mN):
        self.__modelNo=mN
    def getModelNo(self):
        return self.__modelNo
    def countVehicles(self):
        Vehicles.count+=1
        self.assignId(Vehicles.count)
        return Vehicles.count

    def assignId(self,id):
        self.Id=id
    def getId(self):
        return self.Id

v=Vehicles()
print("\nDefault No. of Wheels",v.getNoOfWheels())
print("Default Color:",v.getColor())
print("Default Model No:",v.getModelNo())
print("\nNumber of Objects in v:",v.countVehicles())
print("ID of v:",v.getId())
```

```
v1=Vehicles()
v1.setNoOfWheels(5)
v1.setColor("Black")
v1.setModelNo(2015)
print("\nSet No.of Wheels:",v1.getNoOfWheels())
print("Set Color:",v1.getColor())
print("Set Model No:",v1.getModelNo())
print("\nNumber of Objects in v1:",v1.countVehicles())
print("ID of v1:",v1.getId())
```

IDLE Shell 3.13.14

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/Shaya/Downloads/Inheritance Assignment/Q3.py ========

Default No. of Wheels 4
Default Color: Red
Default Model No: 1987

Number of Objects in v: 1
ID of v: 1

Set No.of Wheels: 5
Set Color: Black
Set Model No: 2015

Number of Objects in v1: 2
ID of v1: 2
>>>
```

Q4. Define a class Fraction which stores a fraction in two instance variables: numerator and denominator. Define appropriate initiator, setters and print methods. Override the following operators: +, -, *, /, >, <, >=, <=, ==, and !=. Add another method simplify to simplify the fraction. You overloaded arithmetic methods should return a simplified fraction.

```
class Fraction():
    def __init__(self, n=0, d=1):
        self.numerator = n
        self.denominator = d

    def setNumerator(self, n):
        self.numerator = n

    def setDenominator(self, d):
        self.denominator = d

    def get(self):
        return self.numerator, self.denominator

    def simplify(self):
        n = self.numerator
        if n < 0:
            n = -n

        c = 1
        for i in range(1, n + 1):
            if self.numerator % i == 0 and self.denominator % i
== 0:
                c = i

        self.numerator = self.numerator // c
        self.denominator = self.denominator // c

    def __str__(self):
        return str(self.numerator) + "/" + str(self.denominator)


    def __add__(self, f2):
        n = self.numerator * f2.denominator + f2.numerator *
self.denominator
        d = self.denominator * f2.denominator
        res = Fraction(n, d)
        res.simplify()
        return res
```

```
    def __sub__(self, f2):
        n = self.numerator * f2.denominator - f2.numerator *
self.denominator
        d = self.denominator * f2.denominator
        res = Fraction(n, d)
        res.simplify()
        return res

    def __mul__(self, f2):
        n = self.numerator * f2.numerator
        d = self.denominator * f2.denominator
        res = Fraction(n, d)
        res.simplify()
        return res

    def __truediv__(self, f2):
        n = self.numerator * f2.denominator
        d = self.denominator * f2.numerator
        res = Fraction(n, d)
        res.simplify()
        return res


    def __gt__(self, f2):
        return (self.numerator * f2.denominator) > (f2.numerator
* self.denominator)

    def __lt__(self, f2):
        return (self.numerator * f2.denominator) < (f2.numerator
* self.denominator)

    def __ge__(self, f2):
        return (self.numerator * f2.denominator) >=
(f2.numerator * self.denominator)

    def __le__(self, f2):
        return (self.numerator * f2.denominator) <=
(f2.numerator * self.denominator)

    def __eq__(self, f2):
        return (self.numerator * f2.denominator) ==
(f2.numerator * self.denominator)

    def __ne__(self, f2):
        return (self.numerator * f2.denominator) !=
(f2.numerator * self.denominator)
```

```
f1 = Fraction()
print("\nDefault Fraction:",f1.get())

f1.setNumerator(2)
f1.setDenominator(4)
print("f1 Set Fraction:",f1.get())

f1.simplify()
print("f1 Simplified Fraction:",f1)

f2 = Fraction()
f2.setNumerator(3)
f2.setDenominator(4)
print("\nf2 Set Fraction:",f2.get())
f2.simplify()
print("Simplied Fraction for f2:",f2)


print("Operator Overloading for Addition:",f1 + f2)
print("Operator Overloading for \'>':",f1 > f2)
```

```
class Fraction():
    def __init__(self, n=0, d=1):
        self.numerator = n
        self.denominator = d

    def setNumerator(self, n):
        self.numerator = n

    def setDenominator(self, d):
        self.denominator = d

    def get(self):
        return self.numerator, self.denominator

    def simplify(self):
        n = self.numerator
        if n < 0:
            n = -n

        c = 1
        for i in range(1, n + 1):
            if self.numerator % i == 0 and self.denominator % i == 0:
                c = i

        self.numerator = self.numerator // c
        self.denominator = self.denominator // c

    def __str__(self):
        return str(self.numerator) + "/" + str(self.denominator)


    def __add__(self, f2):
        n = self.numerator * f2.denominator + f2.numerator * self.denominator
        d = self.denominator * f2.denominator
        res = Fraction(n, d)
        res.simplify()
        return res
```

```
    def __sub__(self, f2):
        n = self.numerator * f2.denominator - f2.numerator * self.denominator
        d = self.denominator * f2.denominator
        res = Fraction(n, d)
        res.simplify()
        return res

    def __mul__(self, f2):
        n = self.numerator * f2.numerator
        d = self.denominator * f2.denominator
        res = Fraction(n, d)
        res.simplify()
        return res

    def __truediv__(self, f2):
        n = self.numerator * f2.denominator
        d = self.denominator * f2.numerator
        res = Fraction(n, d)
        res.simplify()
        return res


    def __gt__(self, f2):
        return (self.numerator * f2.denominator) > (f2.numerator * self.denominator)

    def __lt__(self, f2):
        return (self.numerator * f2.denominator) < (f2.numerator * self.denominator)

    def __ge__(self, f2):
        return (self.numerator * f2.denominator) >= (f2.numerator * self.denominator)

    def __le__(self, f2):
        return (self.numerator * f2.denominator) <= (f2.numerator * self.denominator)

    def __eq__(self, f2):
        return (self.numerator * f2.denominator) == (f2.numerator * self.denominator)
```

```
    def __ne__(self, f2):
        return (self.numerator * f2.denominator) != (f2.numerator * self.denominator)




f1 = Fraction()
print("\nDefault Fraction:",f1.get())

f1.setNumerator(2)
f1.setDenominator(4)
print("f1 Set Fraction:",f1.get())

f1.simplify()
print("f1 Simplified Fraction:",f1)

f2 = Fraction()
f2.setNumerator(3)
f2.setDenominator(4)
print("\nf2 Set Fraction:",f2.get())
f2.simplify()
print("Simplied Fraction for f2:",f2)



print("Operator Overloading for Addition:",f1 + f2)
print("Operator Overloading for \'>':",f1 > f2)
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:\Users\Shaya\Downloads\operator overloading practice question.py =

Default Fraction: (0, 1)
f1 Set Fraction: (2, 4)
f1 Simplified Fraction: 1/2

f2 Set Fraction: (3, 4)
Simplied Fraction for f2: 3/4
Operator Overloading for Addition: 5/4
Operator Overloading for '>': False
>>>
```